

SAGNIK TARAFDAR

141498

LEETCODE:

54. Spiral Matrix

Medium

Topics

Companies

Hint

Given an $m \times n$ matrix, return *all elements of the matrix in spiral order*.

Example 1:

1 →	2 →	3 ↓
4 →	5	6 ↓
↑ 7 ←	8 ←	9

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Example 2:

1 →	2 →	3 →	4 ↓
5 →	6 →	7	8 ↓
↑ 9 ←	10 ←	11 ←	12

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 10
- -100 <= matrix[i][j] <= 100

CODE

class Solution:

def spiralOrder(self, matrix):

res = []

top = 0

bottom = len(matrix) - 1

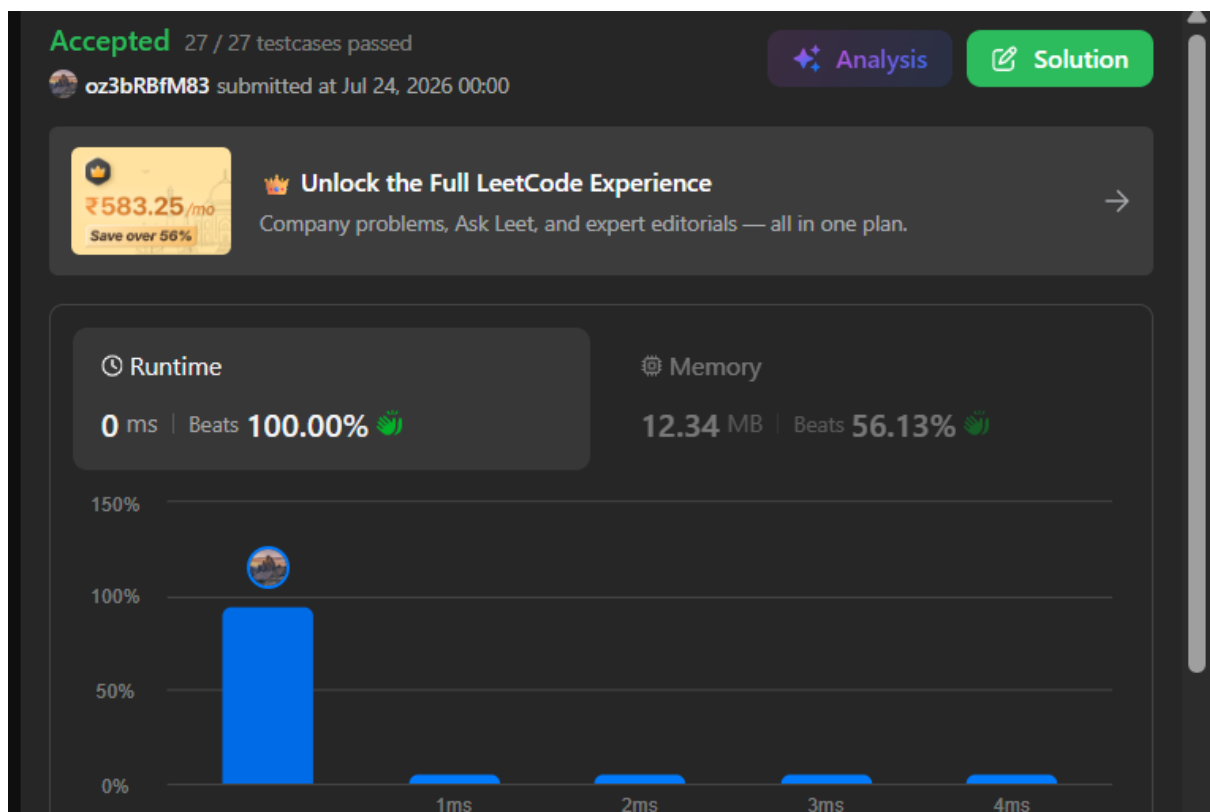
left = 0

right = len(matrix[0]) - 1

while top <= bottom and left <= right:

```
for i in range(left, right + 1):
    res.append(matrix[top][i])
top += 1
for i in range(top, bottom + 1):
    res.append(matrix[i][right])
right -= 1
if top <= bottom:
    for i in range(right, left - 1, -1):
        res.append(matrix[bottom][i])
    bottom -= 1
if left <= right:
    for i in range(bottom, top - 1, -1):
        res.append(matrix[i][left])
    left += 1

return res
```



1041. Robot Bounded In Circle

Medium

Topics

Companies

Hint

On an infinite plane, a robot initially stands at (0, 0) and faces north. Note that:

- The **north direction** is the positive direction of the y-axis.
- The **south direction** is the negative direction of the y-axis.
- The **east direction** is the positive direction of the x-axis.
- The **west direction** is the negative direction of the x-axis.

The robot can receive one of three instructions:

- "G": go straight 1 unit.
- "L": turn 90 degrees to the left (i.e., anti-clockwise direction).

- "R": turn 90 degrees to the right (i.e., clockwise direction).

The robot performs the instructions given in order, and repeats them forever.

Return true if and only if there exists a circle in the plane such that the robot never leaves the circle.

Example 1:

Input: instructions = "GLLGG"

Output: true

Explanation: The robot is initially at (0, 0) facing the north direction.

"G": move one step. Position: (0, 1). Direction: North.

"G": move one step. Position: (0, 2). Direction: North.

"L": turn 90 degrees anti-clockwise. Position: (0, 2). Direction: West.

"L": turn 90 degrees anti-clockwise. Position: (0, 2). Direction: South.

"G": move one step. Position: (0, 1). Direction: South.

"G": move one step. Position: (0, 0). Direction: South.

Repeating the instructions, the robot goes into the cycle: (0, 0) --> (0, 1) --> (0, 2) --> (0, 1) --> (0, 0).

Based on that, we return true.

Example 2:

Input: instructions = "GG"

Output: false

Explanation: The robot is initially at (0, 0) facing the north direction.

"G": move one step. Position: (0, 1). Direction: North.

"G": move one step. Position: (0, 2). Direction: North.

Repeating the instructions, keeps advancing in the north direction and does not go into cycles.

Based on that, we return false.

Example 3:

Input: instructions = "GL"

Output: true

Explanation: The robot is initially at (0, 0) facing the north direction.

"G": move one step. Position: (0, 1). Direction: North.

"L": turn 90 degrees anti-clockwise. Position: (0, 1). Direction: West.

"G": move one step. Position: (-1, 1). Direction: West.

"L": turn 90 degrees anti-clockwise. Position: (-1, 1). Direction: South.

"G": move one step. Position: (-1, 0). Direction: South.

"L": turn 90 degrees anti-clockwise. Position: (-1, 0). Direction: East.

"G": move one step. Position: (0, 0). Direction: East.

"L": turn 90 degrees anti-clockwise. Position: (0, 0). Direction: North.

Repeating the instructions, the robot goes into the cycle: (0, 0) --> (0, 1) --> (-1, 1) --> (-1, 0) --> (0, 0).

Based on that, we return true.

Constraints:

- $1 \leq \text{instructions.length} \leq 100$
- instructions[i] is 'G', 'L' or 'R'.

CODE

class Solution:

```
def isRobotBounded(self, instructions: str) -> bool:
```

```
    x = y = 0
```

```
    direction = 0
```

```
    dirs = [
```

```
(0, 1),  
(1, 0),  
(0, -1),  
(-1, 0)  
]
```

```
for ch in instructions:
```

```
    if ch == 'G':
```

```
        dx, dy = dirs[direction]
```

```
        x += dx
```

```
        y += dy
```

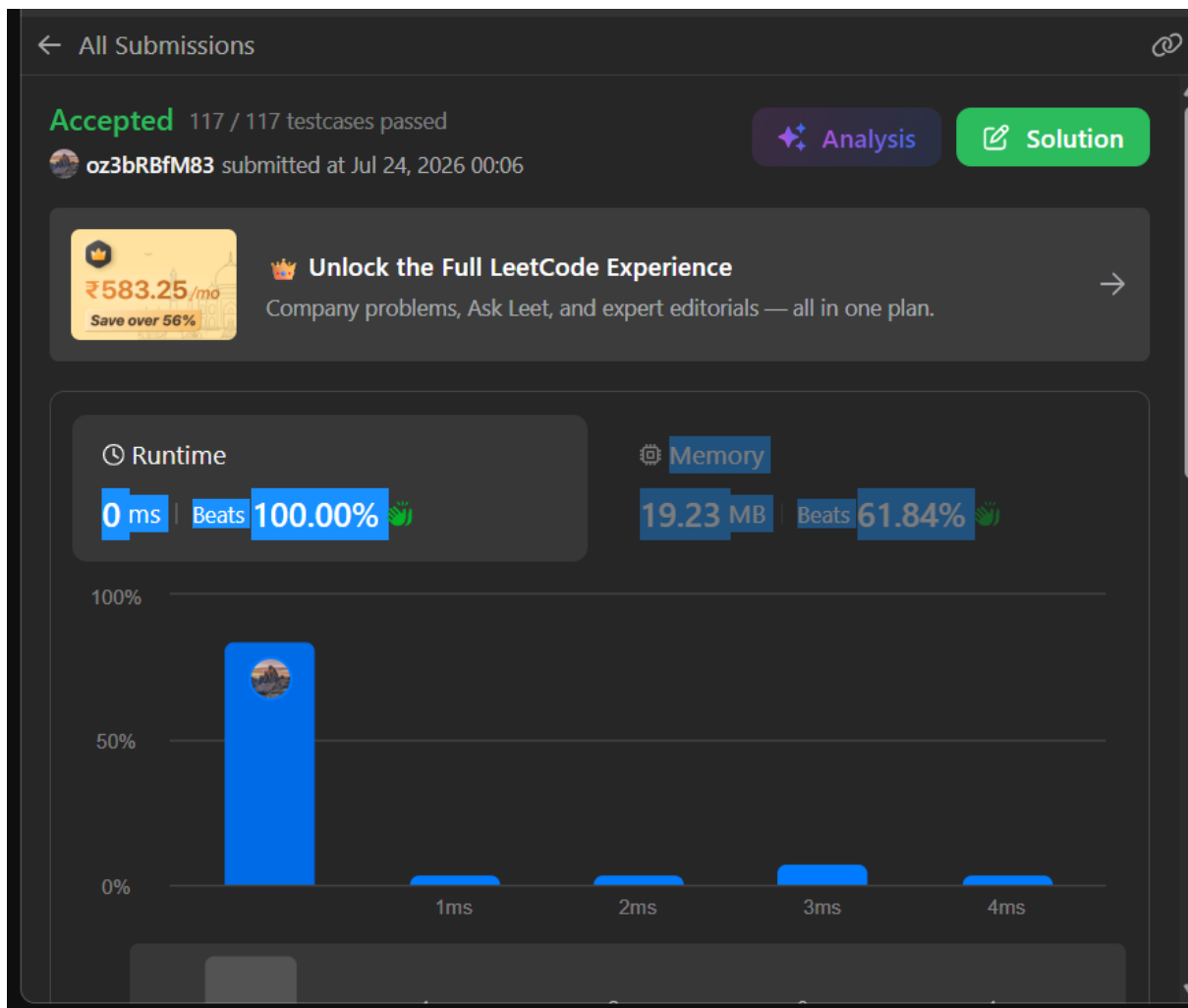
```
    elif ch == 'L':
```

```
        direction = (direction + 3) % 4
```

```
    else:
```

```
        direction = (direction + 1) % 4
```

```
return (x == 0 and y == 0) or direction != 0
```



2. Add Two Numbers

Medium

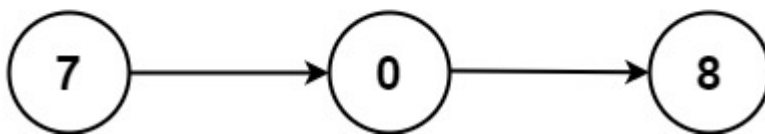
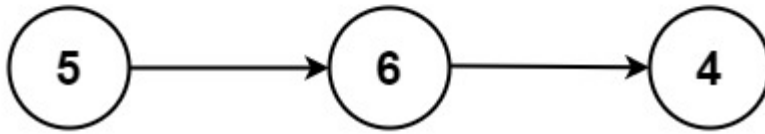
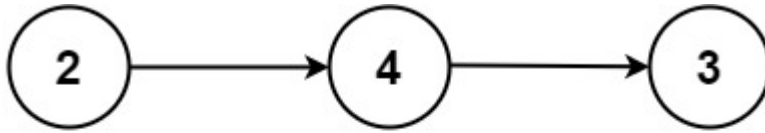
Topics

Companies

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

CODE

```
class Solution:
```

```
    def addTwoNumbers(self, l1, l2):
```

```
        dummy = ListNode(0)
```

```
        current = dummy
```

```
        carry = 0
```

```
        while l1 or l2 or carry:
```

```
            x = l1.val if l1 else 0
```

```
            y = l2.val if l2 else 0
```

```
            total = x + y + carry
```

```
            carry = total // 10
```

```
            current.next = ListNode(total % 10)
```

```
            current = current.next
```

```
        if l1:
```

```
            l1 = l1.next
```

```
        if l2:
```

```
            l2 = l2.next
```

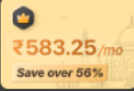
← All Submissions

Accepted 1569 / 1569 testcases passed

oz3bRBfM83 submitted at Jul 24, 2026 00:11

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

8 ms | Beats 13.14%

Memory

19.27 MB | Beats 78.01%

